

Program:

expr.l

```
%{
extern int yylval;
}%
%%
[0-9]+ { yylval=atoi(yytext); return NUMBER; }
\n { return NL ;}
. { return yytext[0]; }
%%
int yywrap()
{
return 1;
}
```

expr.y

```
%{
#include<stdio.h>
#include<stdlib.h>
}%
%token NUMBER NL
%left '+' '-'
%left '*' '/'
%%
stmt : exp NL { printf("Value = %d\n",$1); exit(0); }
exp : exp '+' exp { $$=$1+$3; }
| exp '-' exp { $$=$1-$3; }
| exp '*' exp { $$=$1*$3; }
| exp '/' exp { if($3==0) { printf("Cannot divide by 0"); exit(0); } else $$=$1/$3; }
| '(' exp ')' { $$=$2; }
| NUMBER { $$=$1; }
%%
int yyerror(char *msg)

{
printf("Invalid Expression\n");
exit(0);
}
#include "lex.yy.c"
main ()
{
printf("Enter the expression\n");
yyparse();
}
```

Output:

```

student@compalc12-OptiPlex-3010: ~/22co16
student@compalc12-OptiPlex-3010:~/22co16$ flex expt8.l
student@compalc12-OptiPlex-3010:~/22co16$ bison expt8.y
student@compalc12-OptiPlex-3010:~/22co16$ gcc expt8.tab.c -o calculator -lfl
expt8.tab.c: In function 'yyparse':
expt8.tab.c:1016:16: warning: implicit declaration of function 'yylex' [-Wimplicit-function-declaration]
 1016 |         yychar = yylex ();
      |                   ^~~~~
expt8.tab.c:1193:7: warning: implicit declaration of function 'yyerror'; did you mean 'yyerrok'? [-Wimplicit-function-declaration]
 1193 |         yyerror (YY_("syntax error"));
      |         ^~~~~~
      |         yyerrok
expt8.y: At top level:
expt8.y:24:1: warning: return type defaults to 'int' [-Wimplicit-int]
   24 | main ()
      |     ^~~
student@compalc12-OptiPlex-3010:~/22co16$ ./calculator
Enter the expression
3*5+5-4
Value = 16
student@compalc12-OptiPlex-3010:~/22co16$

```

Conclusion: YACC program to parse an expression for a given grammar was implemented and executed successfully.